

THE EMOTIONAL RELATIONAL MODEL (ERM)

The Ledger That Turns Experience Drivers — Emotionally Weighted — Into Institutional Intelligence

The Relational Operating System for Experience Determinants

Raheem Asghar — December 2025

1. The Missing Data Model

We have relational schemas for money, inventory, identities, commits, packets, logs, metrics, and events.

What we have never had is a **relational model for Experience Drivers** — the system-level determinants of customer satisfaction, dissatisfaction, loyalty, and disloyalty.

Until now.

The **Emotional Relational Model (ERM)** is the categorical leap: the first time Experience Drivers — as inferred from emotionally weighted customer feedback — are formalized as a relational system with enforced invariants at the schema level.

ERM does **not** model raw emotion in isolation.

It governs **experience determinants**, ensuring that emotional signals are:

- anchored to canonical Experience Driver identity
- accumulated into stable, auditable driver state
- preserved with lineage, memory, and causality

Everything else in Decipher lives within this model:

- Experience Drivers define **identity**
- Structured Experience Units (SEUs) hold authoritative **driver state**
- Orchestration Units (OUs) encode **behavioral execution**
- PDAM / PDCA loops, elasticity, temporal intelligence, and memory operate on top of this substrate

ERM is not application logic.

ERM is the **relational enforcement layer** that makes Experience Drivers computable, governable, and institutional.

It is the ground-truth ledger of **Experience Driver state** — not because it computes emotion, but because it ensures that once a driver is identified and stateful, it can never drift, duplicate, or be forgotten.

2. The Three Structural Laws of the ERM

(Invariants enforced exactly like double-entry bookkeeping or UTXO rules)

1 Identity Law

Each **Experience Driver** has exactly **one canonical SEU per analysis window**.

This SEU is its **driver state account**:

the single source of truth for the condition of that experience determinant, emotionally weighted, temporally grounded, and lineage-bound.

No duplicates.

No parallel interpretations.

No competing states.

2 Lineage Law

Every **OU activation** is a timestamped **state transition** linked to a SEU, recording:

- the behavioral intervention
- the execution context
- the observed outcome

Transitions are immutable and fully traceable.

Nothing happens to an **Experience Driver's condition** without being recorded as a state transition.

3 Memory Law

Every Experience Driver state transition retains referential integrity across time.

No state is ever orphaned.

Historical outcomes become constraints and priors for future orchestration.

Experience Drivers do not reset. They accumulate history.

These laws are enforced through relational constraints:

- foreign keys
- uniqueness guarantees
- lineage tables
- state-transition records

Once enforced, Experience Drivers become:

- governable
 - auditable
 - predictable
 - agent-native
 - relationally stable
-

This Is the Experience-Driver Analogue Of

- **Relational model** (Codd)
 - **UTXO ledger** (Satoshi)
 - **Intent model** (Stripe)
 - **Metric + tag model** (Datadog)
-

3. Core Entities and Cardinalities

(The complete ER diagram in nine rows)

All cardinalities are understood per analysis window unless otherwise noted.
Historical records are kept in full for lineage.

No schema changes required

This section is already correct and needs no modification.

Entity	Description	Key Constraints	Cardinality (Core Relationships)
experience_drivers	Canonical list of Experience Drivers	UNIQUE(name) or UNIQUE(external_id)	1 driver → 1 active SEU per window; many SEUs across history. 1 driver → many mentions across time
seus	Structured Experience Unit (experience state account; authoritative state for an Experience Driver within a window, emotionally weighted and temporally grounded)	FK(experience_driver_id) → experience_drivers(id) UNIQUE(experience_driver_id, window_id)	1:1 with a driver in a given window. 1 SEU ← many mentions (all ED mentions in the window collapse into this state)
mentions	Raw unpacked customer mentions associated with the Experience Driver. Used both for SEU aggregation and OU generation via clustering	FK(experience_driver_id) → experience_drivers(id) FK(seu_id) → seus(id)	Many mentions → 1:M potential OUs (via behavioral segmentation: Experience Driver × Feedback Type × Opportunity Stream × Behavioral Signature)
orchestration_units	Mission templates derived from behavioral clusters of mentions; latent missions for the SEU	FK(seu_id) → seus(id)	Storage: 1 SEU → many latent OUs. Generative logic: many mentions (clustered) → 1:M OUs
ou_activations	Executed instances of an Orchestration Unit	FK(orchestration_unit_id) → orchestration_units(id) UNIQUE(orchestration_unit_id, activated_at)	1 OU template → many activation instances over time
problem_statements	Extracted, PDCA-ready problem statement	FK(ou_activation_id) → ou_activations(id) UNIQUE(ou_activation_id)	1 OU activation → 1 problem statement
pdca_cycles	Plan–Do–Check–Act iterations for an activation	FK(problem_statement_id) → problem_statements(id)	1 problem statement → many PDCA cycles
outcome_evaluations	Elasticity and outcome assessment per PDCA cycle	FK(pdca_cycle_id) → pdca_cycles(id) UNIQUE(pdca_cycle_id)	1 PDCA cycle → 1 outcome evaluation

institutional_memory_events	All lineage events (activations, cycles, drifts, silences)	FK(source_entity_id, source_type) → {above tables}	Many-to-many across time; each event points back to one source row
------------------------------------	--	--	--

4. Why the ERM Is the Enabling Substrate — Not the CX Primitive

The **Structured Experience Unit (SEU)** is the atomic primitive of Customer Experience, the smallest unit at which an **Experience Driver** remains identifiable, stateful, and executable.

It is the smallest unit at which a driver remains:

- meaningful
- stateful
- executable
- evaluable

However, a primitive alone is not sufficient. **Primitives require laws.**

The Emotional Relational Model (ERM) is **not** the CX primitive. It is the **relational substrate** that enforces the laws under which SEUs can exist, evolve, and retain memory.

ERM provides:

- identity enforcement
- lineage guarantees
- referential integrity
- temporal continuity

Without ERM, SEUs degrade into disconnected records.

Without SEUs, ERM is an empty ledger.

They are inseparable — but not interchangeable.

This is the inflection point where a system stops being a “CX platform” and becomes infrastructure.

Correct Analogies

Bitcoin

- UTXO = value primitive
- Ledger rules = enforcement substrate

Stripe

- Payment Intent = commerce primitive
- API + constraints = orchestration substrate

Git

- Commit = collaboration primitive
- DAG = integrity + lineage substrate

Snowflake

- Data table = analytical primitive
- Separation of compute/storage = scaling substrate

SEU is the primitive. ERM is the substrate.

5. Properties That Emerge Immediately From the Laws

Property	Enabled By	Enterprise Outcome
Full auditability	Lineage Law	Instant root cause analysis and provable action–outcome trails
Agent-native experience state	Identity Law + Memory Law	Agents query shared state directly; no hallucination risk
Elasticity verification	Referential integrity	Verifiable proof that interventions changed driver state
Silent Guidance	Memory Law	System remembers patterns humans forget
BI compatibility	Pure relational schema	Plug directly into Tableau / PowerBI
Model independence	LLM-agnostic schema	Swap GPT ↔ Claude without rewriting state
Horizontal scalability	Standard relational DB	Billions of rows without redesign
Zero lock-in	Portable SQL schema	Organization owns its Experience Driver ledger

These properties require no additional architecture. They fall directly out of the three laws.

6. PDAM: The Closed Loop Executed Inside the ERM

-- Perceive

INSERT INTO seus (...) -- Identity Law

-- Decide

INSERT INTO orchestration_units (...)

-- Act

INSERT INTO ou_activations (...) -- Lineage Law

-- Memory

INSERT INTO outcome_evaluations (...)

INSERT INTO institutional_memory_events (...)

PDAM is simply a stored procedure over this schema.

7. Final Declaration

This is not another CX platform.

This is the **relational ledger of Experience Drivers** that has been missing since computing began.

Emotion becomes executable **only when embedded as a signal within structured Experience Driver state**.

The Emotional Relational Model is now defined.

The three laws are stable.

The schema is mature.

Every customer-facing institution will eventually operate on this ledger.

Because once **Experience Drivers** gain identity, lineage, and memory enforced at the schema level, themes, dashboards, and sentiment scores become obsolete.

This is not software.

This is infrastructure.

And the ledger is now live.
